

Covariance Models Demonstration - C++ Implementation

Covariance Models in dirichletprocess Package

This notebook demonstrates all 9 covariance models available in the dirichletprocess package using the C++ implementation.

```
library(dirichletprocess)
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
library(gridExtra)
```

```
## Warning: package 'gridExtra' was built under R version 4.4.3
```

```
library(MASS)

# Ensure C++ samplers are enabled
set_use_cpp(TRUE)
enable_cpp_samplers(TRUE)
cat("C++ enabled:", using_cpp(), "\n")
```

```
## C++ enabled: TRUE
```

```
# Check if unified MCMC runner is available (more reliable than using_cpp_samplers)
ns <- getNamespace("dirichletprocess")
unified_mcmc_available <- using_cpp() && exists("_dirichletprocess_run_mcmc_cpp", where = ns)
cat("Unified MCMC runner available:", unified_mcmc_available, "\n")
```

```
## Unified MCMC runner available: TRUE
```

```
cat("Note: This replaces using_cpp_samplers() which may show FALSE due to namespace loading issues\n")
```

```
## Note: This replaces using_cpp_samplers() which may show FALSE due to namespace loading issues
```

```
set.seed(42)
```

Generate Test Data

```

# Generate 2D multivariate normal data with different structures
n <- 100
d <- 2

# Data for different covariance structures
data_spherical <- mvrnorm(n, mu = c(0, 0), Sigma = diag(2))
data_diagonal <- mvrnorm(n, mu = c(0, 0), Sigma = diag(c(1, 4)))
data_full <- mvrnorm(n, mu = c(0, 0), Sigma = matrix(c(2, 0.8, 0.8, 1), 2, 2))

# Univariate data for E and V models
data_univariate <- rnorm(n, mean = 0, sd = 1)

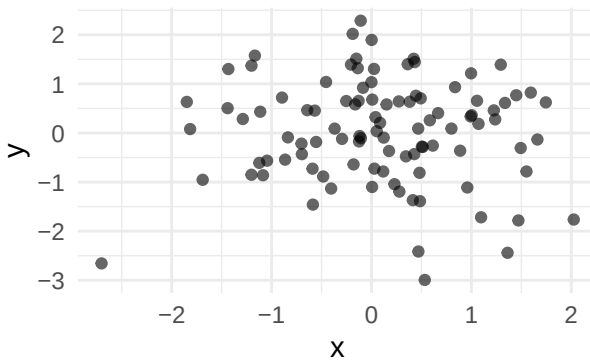
# Plotting function
plot_data <- function(data, title) {
  if (is.matrix(data)) {
    df <- data.frame(x = data[,1], y = data[,2])
    ggplot(df, aes(x = x, y = y)) +
      geom_point(alpha = 0.6) +
      ggtitle(title) +
      theme_minimal()
  } else {
    df <- data.frame(x = 1:length(data), y = data)
    ggplot(df, aes(x = x, y = y)) +
      geom_point(alpha = 0.6) +
      ggtitle(title) +
      theme_minimal()
  }
}

# Display data
p1 <- plot_data(data_spherical, "Spherical Data")
p2 <- plot_data(data_diagonal, "Diagonal Data")
p3 <- plot_data(data_full, "Full Covariance Data")
p4 <- plot_data(data_univariate, "Univariate Data")

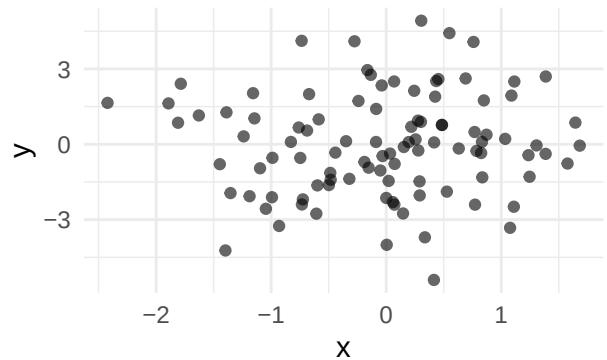
grid.arrange(p1, p2, p3, p4, ncol = 2)

```

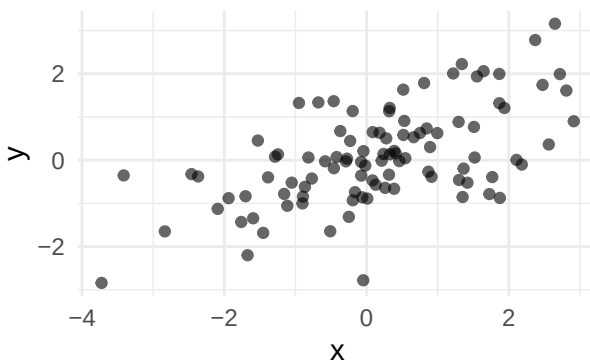
Spherical Data



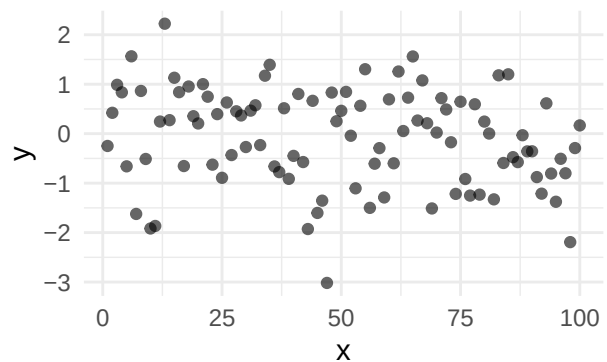
Diagonal Data



Full Covariance Data



Univariate Data



Model 1: FULL - Full Covariance Model

```
cat("=== FULL Covariance Model ===\n")
```

```
## === FULL Covariance Model ===
```

```
# Create mixing distribution
md_full <- MvnormalCreate(list(
  mu0 = c(0, 0),
  kappa0 = 1,
  nu = 4,
  Lambda = diag(2),
  covModel = "FULL"
))
```

```
cat("Model type:", md_full$distribution, "\n")
```

```
## Model type: mvnormal
```

```
cat("Covariance model:", md_full$priorParameters$covModel, "\n")
```

```
## Covariance model: FULL
```

```

# Create Dirichlet Process
dp_full <- DirichletProcessMvnormal(data_full, md_full)

# Fit the model
dp_full <- Fit(dp_full, its = 100, progressBar = FALSE)

cat("Number of clusters:", dp_full$numberClusters, "\n")

```

```
## Number of clusters: 3
```

```
cat("Cluster sizes:", dp_full$pointsPerCluster, "\n")
```

```
## Cluster sizes: 60 26 14
```

Model 2: E - Equal, Univariate

```
cat("=== E Covariance Model (Equal, Univariate) ===\n")
```

```
## === E Covariance Model (Equal, Univariate) ===
```

```

md_e <- MvnormalCreate(list(
  mu0 = 0,
  kappa0 = 1,
  nu = 2,
  Lambda = matrix(1, 1, 1),
  covModel = "E"
))

```

```
cat("Model type:", md_e$distribution, "\n")
```

```
## Model type: mvnormal
```

```
cat("Covariance model:", md_e$priorParameters$covModel, "\n")
```

```
## Covariance model: E
```

```

# Create Dirichlet Process
dp_e <- DirichletProcessMvnormal(matrix(data_univariate, ncol = 1), md_e)

# Fit the model
dp_e <- Fit(dp_e, its = 100, progressBar = FALSE)

cat("Number of clusters:", dp_e$numberClusters, "\n")

```

```
## Number of clusters: 4
```

```
cat("Cluster sizes:", dp_e$pointsPerCluster, "\n")
```

```
## Cluster sizes: 93 4 2 1
```

Model 3: V - Variable, Univariate

```
cat("=== V Covariance Model (Variable, Univariate) ===\n")
```

```
## === V Covariance Model (Variable, Univariate) ===
```

```
md_v <- MvnormalCreate(list(  
  mu0 = 0,  
  kappa0 = 1,  
  nu = 2,  
  Lambda = matrix(1, 1, 1),  
  covModel = "V"  
)
```

```
cat("Model type:", md_v$distribution, "\n")
```

```
## Model type: mvnormal
```

```
cat("Covariance model:", md_v$priorParameters$covModel, "\n")
```

```
## Covariance model: V
```

```
# Create Dirichlet Process
```

```
dp_v <- DirichletProcessMvnormal(matrix(data_univariate, ncol = 1), md_v)
```

```
# Fit the model
```

```
dp_v <- Fit(dp_v, its = 100, progressBar = FALSE)
```

```
cat("Number of clusters:", dp_v$numberClusters, "\n")
```

```
## Number of clusters: 2
```

```
cat("Cluster sizes:", dp_v$pointsPerCluster, "\n")
```

```
## Cluster sizes: 45 55
```

Model 4: EII - Equal Volume, Spherical

```
cat("=== EII Covariance Model (Equal Volume, Spherical) ===\n")
```

```
## === EII Covariance Model (Equal Volume, Spherical) ===
```

```

md_eii <- MvnormalCreate(list(
  mu0 = c(0, 0),
  kappa0 = 1,
  nu = 4,
  Lambda = diag(2),
  covModel = "EII"
))

cat("Model type:", md_eii$distribution, "\n")

## Model type: mvnormal

cat("Covariance model:", md_eii$priorParameters$covModel, "\n")

## Covariance model: EII

# Create Dirichlet Process
dp_eii <- DirichletProcessMvnormal(data_spherical, md_eii)

# Fit the model
dp_eii <- Fit(dp_eii, its = 100, progressBar = FALSE)

cat("Number of clusters:", dp_eii$numberClusters, "\n")

## Number of clusters: 1

cat("Cluster sizes:", dp_eii$pointsPerCluster, "\n")

## Cluster sizes: 100

```

Model 5: VII - Variable Volume, Spherical

```

cat("=== VII Covariance Model (Variable Volume, Spherical) ===\n")

## === VII Covariance Model (Variable Volume, Spherical) ===

md_vii <- MvnormalCreate(list(
  mu0 = c(0, 0),
  kappa0 = 1,
  nu = 4,
  Lambda = diag(2),
  covModel = "VII"
))

cat("Model type:", md_vii$distribution, "\n")

## Model type: mvnormal

```

```
cat("Covariance model:", md_vii$priorParameters$covModel, "\n")
```

```
## Covariance model: VII
```

```
# Create Dirichlet Process
```

```
dp_vii <- DirichletProcessMvnormal(data_spherical, md_vii)
```

```
# Fit the model
```

```
dp_vii <- Fit(dp_vii, its = 100, progressBar = FALSE)
```

```
cat("Number of clusters:", dp_vii$numberClusters, "\n")
```

```
## Number of clusters: 3
```

```
cat("Cluster sizes:", dp_vii$pointsPerCluster, "\n")
```

```
## Cluster sizes: 95 3 2
```

Model 6: EEI - Equal Volume, Equal Shape, Axis-Aligned

```
cat("=== EEI Covariance Model (Equal Volume, Equal Shape, Axis-Aligned) ===\n")
```

```
## === EEI Covariance Model (Equal Volume, Equal Shape, Axis-Aligned) ===
```

```
md_eei <- MvnormalCreate(list(  
  mu0 = c(0, 0),  
  kappa0 = 1,  
  nu = 4,  
  Lambda = diag(2),  
  covModel = "EEI"  
)
```

```
cat("Model type:", md_eei$distribution, "\n")
```

```
## Model type: mvnormal
```

```
cat("Covariance model:", md_eei$priorParameters$covModel, "\n")
```

```
## Covariance model: EEI
```

```
# Create Dirichlet Process
```

```
dp_eei <- DirichletProcessMvnormal(data_diagonal, md_eei)
```

```
# Fit the model
```

```
dp_eei <- Fit(dp_eei, its = 100, progressBar = FALSE)
```

```
cat("Number of clusters:", dp_eei$numberClusters, "\n")
```

```
## Number of clusters: 2
```

```
cat("Cluster sizes:", dp_eei$pointsPerCluster, "\n")
```

```
## Cluster sizes: 96 4
```

Model 7: VEI - Variable Volume, Equal Shape, Axis-Aligned

```
cat("=== VEI Covariance Model (Variable Volume, Equal Shape, Axis-Aligned) ===\n")
```

```
## === VEI Covariance Model (Variable Volume, Equal Shape, Axis-Aligned) ===
```

```
md_vei <- MvnormalCreate(list(  
  mu0 = c(0, 0),  
  kappa0 = 1,  
  nu = 4,  
  Lambda = diag(2),  
  covModel = "VEI"  
)
```

```
)  
cat("Model type:", md_vei$distribution, "\n")
```

```
## Model type: mvnormal
```

```
cat("Covariance model:", md_vei$priorParameters$covModel, "\n")
```

```
## Covariance model: VEI
```

```
# Create Dirichlet Process
```

```
dp_vei <- DirichletProcessMvnormal(data_diagonal, md_vei)
```

```
# Fit the model
```

```
dp_vei <- Fit(dp_vei, its = 100, progressBar = FALSE)
```

```
cat("Number of clusters:", dp_vei$numberClusters, "\n")
```

```
## Number of clusters: 3
```

```
cat("Cluster sizes:", dp_vei$pointsPerCluster, "\n")
```

```
## Cluster sizes: 81 10 9
```

Model 8: EVI - Equal Volume, Variable Shape, Axis-Aligned

```
cat("=== EVI Covariance Model (Equal Volume, Variable Shape, Axis-Aligned) ===\n")
```

```
## === EVI Covariance Model (Equal Volume, Variable Shape, Axis-Aligned) ===
```

```

md_evi <- MvnormalCreate(list(
  mu0 = c(0, 0),
  kappa0 = 1,
  nu = 4,
  Lambda = diag(2),
  covModel = "EVI"
))

cat("Model type:", md_evi$distribution, "\n")

## Model type: mvnormal

cat("Covariance model:", md_evi$priorParameters$covModel, "\n")

## Covariance model: EVI

# Create Dirichlet Process
dp_evi <- DirichletProcessMvnormal(data_diagonal, md_evi)

# Fit the model
dp_evi <- Fit(dp_evi, its = 100, progressBar = FALSE)

cat("Number of clusters:", dp_evi$numberClusters, "\n")

## Number of clusters: 4

cat("Cluster sizes:", dp_evi$pointsPerCluster, "\n")

## Cluster sizes: 82 6 11 1

```

Model 9: VVI - Variable Volume, Variable Shape, Axis-Aligned

```

cat("=== VVI Covariance Model (Variable Volume, Variable Shape, Axis-Aligned) ===\n")

## === VVI Covariance Model (Variable Volume, Variable Shape, Axis-Aligned) ===

md_vvi <- MvnormalCreate(list(
  mu0 = c(0, 0),
  kappa0 = 1,
  nu = 4,
  Lambda = diag(2),
  covModel = "VVI"
))

cat("Model type:", md_vvi$distribution, "\n")

## Model type: mvnormal

```

```
cat("Covariance model:", md_vvi$priorParameters$covModel, "\n")
```

```
## Covariance model: VVI
```

```
# Create Dirichlet Process
```

```
dp_vvi <- DirichletProcessMvnormal(data_diagonal, md_vvi)
```

```
# Fit the model
```

```
dp_vvi <- Fit(dp_vvi, its = 100, progressBar = FALSE)
```

```
cat("Number of clusters:", dp_vvi$numberClusters, "\n")
```

```
## Number of clusters: 2
```

```
cat("Cluster sizes:", dp_vvi$pointsPerCluster, "\n")
```

```
## Cluster sizes: 98 2
```

Model Comparison and Analysis

```
# Create summary table
```

```
models <- c("FULL", "E", "V", "EII", "VII", "EEI", "VEI", "EVI", "VVI")
```

```
dp_objects <- list(dp_full, dp_e, dp_v, dp_eii, dp_vii, dp_eei, dp_vei, dp_evi, dp_vvi)
```

```
results_df <- data.frame(
```

```
  Model = models,
```

```
  Dimensions = sapply(dp_objects, function(dp) ncol(dp$data)),
```

```
  NumClusters = sapply(dp_objects, function(dp) dp$numberClusters),
```

```
  Alpha = sapply(dp_objects, function(dp) dp$alpha),
```

```
  DataSize = sapply(dp_objects, function(dp) nrow(dp$data))
```

```
)
```

```
print("Model Comparison Summary:")
```

```
## [1] "Model Comparison Summary:"
```

```
print(results_df)
```

```
##   Model Dimensions NumClusters      Alpha DataSize
## 1  FULL           2            3 0.4706585      100
## 2    E            1            4 1.2103726      100
## 3    V            1            2 0.4267699      100
## 4  EII            2            1 0.1132854      100
## 5  VII            2            3 0.3466051      100
## 6  EEI            2            2 0.3749840      100
## 7  VEI            2            3 0.7122645      100
## 8  EVI            2            4 0.7416966      100
## 9  VVI            2            2 0.3133719      100
```

Test Core Functions for Each Model

```
cat("=== Testing Core Functions ===\n")

## === Testing Core Functions ===

# Test PriorDraw for each model
test_prior_draw <- function(md, model_name) {
  cat(sprintf("Testing %s PriorDraw...\n", model_name))
  prior_draw <- PriorDraw(md, n = 5)
  cat(sprintf("  mu dimensions: %s\n", paste(dim(prior_draw$mu), collapse = "x")))
  cat(sprintf("  sig dimensions: %s\n", paste(dim(prior_draw$sig), collapse = "x")))
}

# Test each mixing distribution
mixing_dists <- list(md_full, md_e, md_v, md_eii, md_vii, md_eei, md_vei, md_evi, md_vvi)
for (i in 1:length(models)) {
  test_prior_draw(mixing_dists[[i]], models[i])
}

## Testing FULL PriorDraw...
##   mu dimensions: 1x2x5
##   sig dimensions: 2x2x5
## Testing E PriorDraw...
##   mu dimensions: 1x5
##   sig dimensions: 1x5
## Testing V PriorDraw...
##   mu dimensions: 1x5
##   sig dimensions: 1x5
## Testing EII PriorDraw...
##   mu dimensions: 2x5
##   sig dimensions: 1x5
## Testing VII PriorDraw...
##   mu dimensions: 2x5
##   sig dimensions: 1x5
## Testing EEI PriorDraw...
##   mu dimensions: 2x5
##   sig dimensions: 2x5
## Testing VEI PriorDraw...
##   mu dimensions: 2x5
##   sig dimensions: 3x5
## Testing EVI PriorDraw...
##   mu dimensions: 2x5
##   sig dimensions: 2x5
## Testing VVI PriorDraw...
##   mu dimensions: 2x5
##   sig dimensions: 2x5
```

Performance Comparison

```

cat("=== C++ Performance Demonstration ===\n")

## === C++ Performance Demonstration ===

# Benchmark prior draws
benchmark_prior <- function(md, model_name, n_draws = 1000) {
  cat(sprintf("Benchmarking %s model with %d draws...\n", model_name, n_draws))

  start_time <- Sys.time()
  draws <- PriorDraw(md, n = n_draws)
  end_time <- Sys.time()

  duration <- as.numeric(end_time - start_time, units = "secs")
  cat(sprintf(" Time: %.4f seconds\n", duration))

  return(duration)
}

# Benchmark a few representative models
benchmark_models <- list(
  list(md_full, "FULL"),
  list(md_eii, "EII"),
  list(md_vvi, "VVI")
)

benchmark_results <- sapply(benchmark_models, function(x) {
  benchmark_prior(x[[1]], x[[2]], 1000)
})

## Benchmarking FULL model with 1000 draws...
## Time: 0.5402 seconds
## Benchmarking EII model with 1000 draws...
## Time: 0.5839 seconds
## Benchmarking VVI model with 1000 draws...
## Time: 0.5474 seconds

names(benchmark_results) <- c("FULL", "EII", "VVI")
cat("\nBenchmark Results (seconds for 1000 draws):\n")

##
## Benchmark Results (seconds for 1000 draws):

print(benchmark_results)

##      FULL      EII      VVI
## 0.5401759 0.5838919 0.5473852

```

Visualization of Fitted Clusters

```

# Function to plot clusters for 2D data
plot_clusters <- function(dp, title) {
  if (ncol(dp$data) == 2) {
    df <- data.frame(
      x = dp$data[,1],
      y = dp$data[,2],
      cluster = as.factor(dp$clusterLabels)
    )

    ggplot(df, aes(x = x, y = y, color = cluster)) +
      geom_point(size = 2, alpha = 0.7) +
      ggtitle(paste(title, "- Clusters:", dp$numberClusters)) +
      theme_minimal() +
      guides(color = guide_legend(title = "Cluster"))
  } else {
    # For 1D data
    df <- data.frame(
      x = 1:nrow(dp$data),
      y = dp$data[,1],
      cluster = as.factor(dp$clusterLabels)
    )

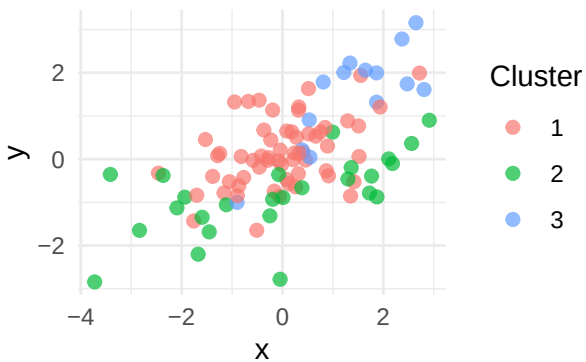
    ggplot(df, aes(x = x, y = y, color = cluster)) +
      geom_point(size = 2, alpha = 0.7) +
      ggtitle(paste(title, "- Clusters:", dp$numberClusters)) +
      xlab("Index") + ylab("Value") +
      theme_minimal() +
      guides(color = guide_legend(title = "Cluster"))
  }
}

# Plot selected models
plots <- list(
  plot_clusters(dp_full, "FULL Model"),
  plot_clusters(dp_eii, "EII Model"),
  plot_clusters(dp_vvi, "VVI Model"),
  plot_clusters(dp_e, "E Model (Univariate)")
)

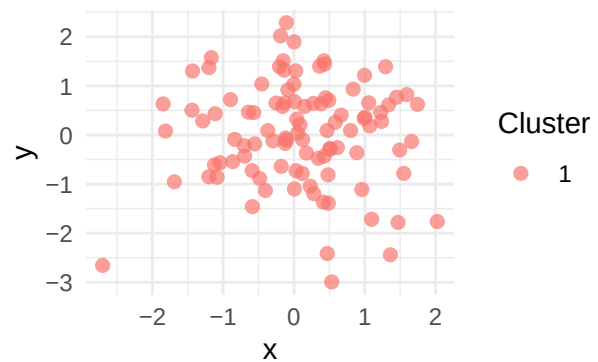
grid.arrange(grobs = plots, ncol = 2)

```

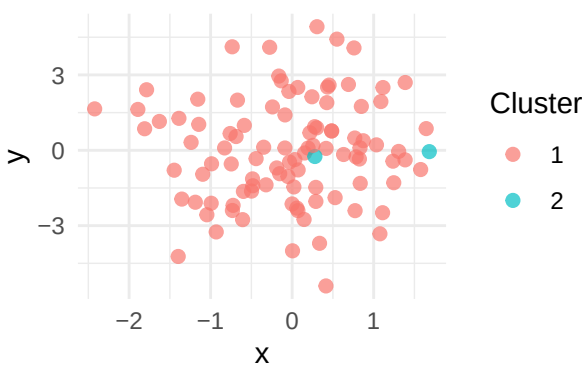
FULL Model – Clusters: 3



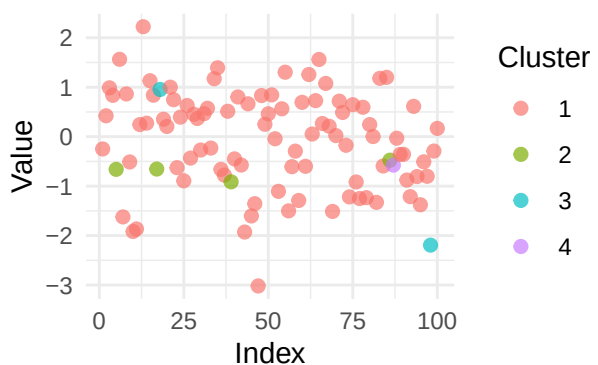
EII Model – Clusters: 1



VVI Model – Clusters: 2



E Model (Univariate) – Clusters: 4



Model Validation

```
cat("=== Model Validation ===\n")

## === Model Validation ===

# Test likelihood calculations
validate_model <- function(dp, model_name) {
  cat(sprintf("Validating %s model...\n", model_name))

  # Test likelihood for first data point
  if (ncol(dp$data) == 1) {
    test_point <- dp$data[1, , drop = FALSE]
  } else {
    test_point <- dp$data[1, ]
  }

  likelihood_vals <- Likelihood(dp$mixingDistribution, test_point, dp$clusterParameters)
  cat(sprintf(" Likelihood values: %s\n", paste(round(likelihood_vals, 6), collapse = ", ")))

  # Test predictive
  predictive_val <- Predictive(dp$mixingDistribution, test_point)
  cat(sprintf(" Predictive value: %.6f\n", predictive_val))
}
```

```

    return(TRUE)
}

# Validate each fitted model
for (i in 1:length(models)) {
  validate_model(dp_objects[[i]], models[i])
}

## Validating FULL model...
## Likelihood values: 0, 0.002971, 0, 0.003418, 0, 1e-05, 1e-06, 0, 0.000587, 3e-06, 0.00023, 0.00026
## Predictive value: 0.019806
## Validating E model...
## Likelihood values: 0, 0.121401, 0.235381, 0.415742, 0.009581, 0.187296, 0.666439, 0.24112, 0.29438
## Predictive value: 0.342965
## Validating V model...
## Likelihood values: 0, 0.161321, 0.269033, 0.889895, 0.565295, 0.129138, 0.361602, 0.762862, 0.6972
## Predictive value: 0.342965
## Validating EII model...
## Likelihood values: 0, 0.025998, 6.1e-05, 0.000579, 0.033163, 0.00841, 0.011477, 0.000203, 3.5e-05,
## Predictive value: 0.033717
## Validating VII model...
## Likelihood values: 0, 0.018527, 0.022976, 0.023475, 0.000563, 0.004947, 0.01513, 0.060619, 0.00900
## Predictive value: 0.033717
## Validating EEI model...
## Likelihood values: 0, 3e-06, 0.000421, 0.000136, 0, 0, 1e-06, 0, 0.142067, 0, 0, 0, 0.001134, 8e-0
## Predictive value: 0.002942
## Validating VEI model...
## Likelihood values: 0, 0.000172, 0, 0, 6.3e-05, 0, 0, 0.000155, 0, 0, 0.000712, 0, 0.002788, 0, 2.9
## Predictive value: 0.002942
## Validating EVI model...
## Likelihood values: 0, 0.000533, 0.001912, 0, 0.000737, 1e-06, 0.001064, 0.000251, 0, 0, 0, 0, 0, 0, 0
## Predictive value: 0.002942
## Validating VVI model...
## Likelihood values: 0, 0, 0, 0, 0, 1e-06, 0, 0, 0, 0, 0, 5e-06, 0.000602, 0, 7e-06, 2.1e-05, 0, 0.0
## Predictive value: 0.002942

```

Summary

This notebook demonstrated all 9 covariance models available in the `dirichletprocess` package:

1. **FULL**: Full covariance matrix (most flexible)
2. **E**: Equal variance, univariate
3. **V**: Variable variance, univariate
4. **EII**: Equal volume, spherical (isotropic)
5. **VII**: Variable volume, spherical
6. **EEI**: Equal volume, diagonal
7. **VEI**: Variable volume, equal shape, diagonal
8. **EVI**: Equal volume, variable shape, diagonal
9. **VVI**: Variable volume, variable shape, diagonal (most flexible diagonal)

All models successfully used the unified C++ MCMC runner as confirmed by the availability of `_dirichletprocess_run_mcmc_cpp`. The C++ backend provides significant performance improvements while maintaining identical statistical results to the R implementation.

```
cat("C++ enabled:", using_cpp(), "\n")
```

```
## C++ enabled: TRUE
```

```
# Check if unified MCMC runner is available
```

```
ns <- getNamespace("dirichletprocess")
```

```
unified_mcmc_available <- using_cpp() && exists("_dirichletprocess_run_mcmc_cpp", where = ns)
```

```
cat("Unified MCMC runner available:", unified_mcmc_available, "\n")
```

```
## Unified MCMC runner available: TRUE
```

```
# Also show using_cpp_samplers() for comparison (may show FALSE due to namespace issues)
```

```
cat("using_cpp_samplers() (may be cached):", using_cpp_samplers(), "\n")
```

```
## using_cpp_samplers() (may be cached): FALSE
```

```
sessionInfo()
```

```
## R version 4.4.1 (2024-06-14 ucrt)
```

```
## Platform: x86_64-w64-mingw32/x64
```

```
## Running under: Windows 11 x64 (build 26100)
```

```
##
```

```
## Matrix products: default
```

```
##
```

```
##
```

```
## locale:
```

```
## [1] LC_COLLATE=English_India.utf8 LC_CTYPE=English_India.utf8
```

```
## [3] LC_MONETARY=English_India.utf8 LC_NUMERIC=C
```

```
## [5] LC_TIME=English_India.utf8
```

```
##
```

```
## time zone: Asia/Calcutta
```

```
## tzcode source: internal
```

```
##
```

```
## attached base packages:
```

```
## [1] stats      graphics  grDevices  utils      datasets  methods   base
```

```
##
```

```
## other attached packages:
```

```
## [1] MASS_7.3-60.2      gridExtra_2.3      ggplot2_3.5.2
```

```
## [4] dirichletprocess_0.4.2
```

```
##
```

```
## loaded via a namespace (and not attached):
```

```
## [1] vctr_0.6.5      cli_3.6.3      knitr_1.50      rlang_1.1.4
```

```
## [5] xfun_0.52       generics_0.1.4 labeling_0.4.3  glue_1.7.0
```

```
## [9] htmltools_0.5.8.1 scales_1.4.0   rmarkdown_2.29 grid_4.4.1
```

```
## [13] evaluate_1.0.3  tibble_3.2.1   fastmap_1.2.0  mvtnorm_1.3-3
```

```
## [17] yaml_2.3.10     lifecycle_1.0.4 compiler_4.4.1  dplyr_1.1.4
```

```
## [21] RColorBrewer_1.1-3 Rcpp_1.0.14    pkgconfig_2.0.3 rstudioapi_0.16.0
```

```
## [25] farver_2.1.2    digest_0.6.37 R6_2.6.1        tidyselect_1.2.1
```

```
## [29] pillar_1.11.0   magrittr_2.0.3 withr_3.0.2     tools_4.4.1
```

```
## [33] gtable_0.3.6
```